

# AWS GuardDuty Threat Detection Lab

Cloud Threat Detection — Finding Investigation — SOC Response

Project Date: August 17, 2026 | Author: Houston Jones

---

## 1. Project Overview

This project demonstrates AWS GuardDuty — Amazon's native cloud threat detection service. GuardDuty continuously monitors AWS accounts for malicious activity and unauthorized behavior by analyzing CloudTrail logs, VPC Flow Logs, and DNS logs automatically. This lab covers enabling GuardDuty, generating and investigating findings, and documenting SOC response procedures for two critical finding types.

| Field                 | Details                  |
|-----------------------|--------------------------|
| Service               | AWS GuardDuty            |
| Account ID            | 897722702309             |
| Region                | us-east-1                |
| Trial Period          | 30 days free             |
| Findings Generated    | 410 sample findings      |
| Findings Investigated | 2 High severity findings |
| Date                  | August 17, 2026          |

## 2. What is AWS GuardDuty

AWS GuardDuty is a managed threat detection service that continuously monitors AWS accounts and workloads for malicious activity. Unlike traditional security tools that require agent installation and infrastructure management, GuardDuty operates entirely at the AWS service level — analyzing three primary data sources:

| Data Source    | What It Monitors                                   |
|----------------|----------------------------------------------------|
| AWS CloudTrail | API calls, console logins, and account activity    |
| VPC Flow Logs  | Network traffic patterns between AWS resources     |
| DNS Logs       | Domain name resolution requests from EC2 instances |

GuardDuty uses machine learning, anomaly detection, and integrated threat intelligence from AWS, CrowdStrike, and Proofpoint to identify threats that would be difficult to detect manually.

### 3. GuardDuty Finding Categories

| Category            | Description                                                    | Example                         |
|---------------------|----------------------------------------------------------------|---------------------------------|
| Backdoor            | EC2 instance communicating with known malicious infrastructure | C&C server communication        |
| Behavior            | Unusual API activity or network behavior                       | Unusual traffic patterns        |
| CryptoCurrency      | EC2 instance mining cryptocurrency                             | Bitcoin mining activity         |
| PenTest             | Penetration testing tools detected                             | Kali Linux tools running        |
| Recon               | Reconnaissance activity against AWS resources                  | Port scanning, user enumeration |
| Trojan              | EC2 instance exhibiting trojan-like behavior                   | DNS rebinding attacks           |
| UnauthorizedAccess  | Credentials or access keys used from unusual locations         | Login from Tor exit node        |
| CredentialAccess    | Compromised credentials being used                             | Stolen IAM key usage            |
| PrivilegeEscalation | Attempts to gain higher privileges                             | Unusual IAM policy changes      |

### 4. Finding Investigation 1 — EC2 Command & Control

#### 4.1 Finding Summary

| Field                | Value                            |
|----------------------|----------------------------------|
| Finding ID           | 6de8473231b24622a8426316c833f62e |
| Type                 | Backdoor:EC2/C&CActivity.B       |
| Severity             | HIGH                             |
| Region               | us-east-1                        |
| Account ID           | 897722702309                     |
| Resource Type        | EC2 Instance                     |
| Instance ID          | i-999999999                      |
| Created              | 08-17-2026 11:15:56              |
| Action Type          | NETWORK_CONNECTION               |
| Connection Direction | OUTBOUND                         |
| Protocol             | TCP                              |

| Field   | Value |
|---------|-------|
| Blocked | false |

## 4.2 Network Details

| Field                      | Value                   |
|----------------------------|-------------------------|
| Local IP                   | 10.0.0.23               |
| Public IP                  | 198.51.100.1            |
| Network Interface          | eni-abcdef00            |
| Destination IP (Malicious) | 198.51.100.0            |
| Destination Port           | 25 (SMTP)               |
| Port Name                  | SMTP                    |
| Threat List                | AWS Threat Intelligence |

## 4.3 Analysis

This finding indicates an EC2 instance is making outbound TCP connections on port 25 (SMTP) to a known Command and Control server at IP 198.51.100.0. This is a critical security event for several reasons:

- Port 25 (SMTP) is blocked by AWS by default for most accounts — any outbound SMTP traffic is highly suspicious
- The destination IP appears on AWS threat intelligence feeds as a known malicious C2 server
- Outbound C2 communication indicates the instance has been compromised and malware is actively communicating with an attacker-controlled server
- The connection was NOT blocked — meaning the malicious traffic was successfully leaving the environment
- This pattern is consistent with a botnet infection, ransomware C2 channel, or spam relay

## 4.4 SOC Response Procedures

### Immediate Actions (Within 15 minutes):

- Isolate the EC2 instance by modifying its security group to block all outbound traffic except to AWS management endpoints
- Take an EBS snapshot of the instance for forensic preservation before any changes
- Block the malicious IP 198.51.100.0 in the VPC Network ACL

### Investigation Actions (Within 1 hour):

- Query CloudTrail for all API calls made from this instance in the past 30 days
- Check VPC Flow Logs for the full scope of outbound connections — how long has this been happening?
- Identify the process making the SMTP connection using EC2 Systems Manager Session Manager
- Check for any data exfiltration — what data could the instance have accessed?

#### Remediation Actions:

- Terminate the compromised instance after forensic snapshot is complete
- Launch a replacement instance from a known-good AMI
- Rotate any IAM credentials or secrets the instance had access to
- Conduct root cause analysis — how was the instance compromised?
- Implement IMDSv2 to prevent SSRF-based credential theft

## 5. Finding Investigation 2 — Compromised IAM Credentials

### 5.1 Finding Summary

| Field         | Value                                           |
|---------------|-------------------------------------------------|
| Finding ID    | 16ea2cd111264117a8ae5f21a657e360                |
| Type          | CredentialAccess:IAMUser/CompromisedCredentials |
| Severity      | HIGH                                            |
| Region        | us-east-1                                       |
| Account ID    | 897722702309                                    |
| Resource Type | IAM Access Key                                  |
| Access Key ID | GeneratedFindingAccessKeyId                     |
| User Name     | GeneratedFindingUserName                        |
| User Type     | IAMUser                                         |
| Created       | 08-17-2026 11:15:56                             |
| Action Type   | AWS_API_CALL                                    |
| API Called    | GetCallerIdentity                               |
| Error Code    | AccessDenied                                    |

### 5.2 Actor Details

| Field          | Value                          |
|----------------|--------------------------------|
| Attacker IP    | 198.51.100.0                   |
| Caller Type    | Remote IP                      |
| API Call Count | 1                              |
| Service Name   | GeneratedFindingAPIServiceName |

### 5.3 Analysis

This finding indicates a compromised IAM access key was used by an attacker to call the `GetCallerIdentity` API. This is extremely significant:

- `GetCallerIdentity` is the first API call attackers make after stealing credentials — it reveals the AWS account ID, user ARN, and assumed role. This is reconnaissance to understand what they have compromised.
- The call returned `AccessDenied` — the attacker was blocked from this specific action but the credential theft itself was successful
- The call originated from IP 198.51.100.0 which is identified as malicious by AWS threat intelligence
- IAM access keys are commonly stolen through exposed GitHub repositories, phishing attacks, or compromised developer workstations
- Even though this call was denied the attacker may have successfully made other API calls before or after this event

### 5.4 SOC Response Procedures

#### Immediate Actions (Within 15 minutes):

- Immediately disable the compromised access key in IAM — do not delete it yet as it may be needed for forensic investigation
- Create a new access key for the legitimate user if needed for business continuity
- Add an explicit Deny policy to the user blocking all actions until investigation is complete

#### Investigation Actions (Within 1 hour):

- Query CloudTrail for ALL API calls made with this access key — `GetCallerIdentity` may not be the only call
- Look for successful API calls that may have created resources, exfiltrated data, or escalated privileges
- Check if any new IAM users, roles, or access keys were created using the compromised key
- Check if any S3 buckets were accessed or made public
- Identify how the key was leaked — search GitHub, check developer workstations

#### Remediation Actions:

- Delete the compromised access key after forensic investigation is complete
- Implement MFA for all IAM users
- Enable AWS Config to detect IAM policy changes
- Implement least privilege — ensure access keys only have permissions required for their function
- Consider using IAM roles instead of long-term access keys where possible

## 6. GuardDuty Finding Severity Matrix

| Severity | Score     | Response Time                 | Action Required                                                      |
|----------|-----------|-------------------------------|----------------------------------------------------------------------|
| High     | 7.0 - 8.9 | Immediate — within 15 minutes | Isolate resource, begin investigation, escalate to security engineer |
| Medium   | 4.0 - 6.9 | Within 1 hour                 | Investigate and determine if escalation is needed                    |
| Low      | 1.0 - 3.9 | Within 24 hours               | Review and document — may indicate reconnaissance or failed attacks  |

## 7. Key Takeaways

- GuardDuty requires zero infrastructure to deploy — it analyzes existing AWS log sources automatically
- High severity findings require immediate response — C2 communication and compromised credentials are critical incidents
- GetCallerIdentity is the attacker's first move after stealing IAM credentials — always investigate what came before and after
- GuardDuty findings should be correlated with CloudTrail logs for complete incident investigation
- Isolation before investigation — always preserve forensic evidence before remediating
- GuardDuty integrates with AWS Security Hub, EventBridge, and Lambda for automated response workflows

## 8. Integration with Existing Security Stack

GuardDuty complements the existing AWS security monitoring pipeline built in previous labs:

| Service    | Role in Security Stack                                   |
|------------|----------------------------------------------------------|
| CloudTrail | Raw API call logging — GuardDuty's primary data source   |
| S3         | Long term log storage and forensic evidence preservation |

| Service    | Role in Security Stack                                        |
|------------|---------------------------------------------------------------|
| CloudWatch | Metric based alerting and operational monitoring              |
| GuardDuty  | Intelligent threat detection using ML and threat intelligence |
| SNS        | Alert notification delivery to security team                  |